

# RESEARCH PAPER

RESQNET: RISK-AWARE GRAPH  
OPTIMIZATION FOR EMERGENCY  
ROUTING IN DISASTER  
ENVIRONMENTS

GITHUB



DEMO LINK



PREPARED BY  
VIREN SINGH

DOI: [HTTPS://DOI.ORG/10.5281/ZENODO.20658065](https://doi.org/10.5281/ZENODO.20658065)

Gmail:- [thakurvirensingh1@gmail.com](mailto:thakurvirensingh1@gmail.com)

Demo link: <https://virensingh011.github.io/ResQnet/>



# TITLE PAGE

Author: Viren Singh

Gmail: [thakurvirensingh1@gmail.com](mailto:thakurvirensingh1@gmail.com)

DOI: <https://doi.org/10.5281/zenodo.20658065>

Demo link:

<https://virensingh011.github.io/ResQnet/>

..... COPYRIGHT (C) 2026 VIREN SINGH. ALL RIGHTS RESERVED. ....

**GITHUB**



**DEMO LINK**



KINDLY NOTE

THIS PROJECT WAS MADE INDEPENDENTLY

Conventional routing systems like Google Maps and Waze optimize for shortest distance or fastest travel time under normal conditions. During disasters—earthquakes, floods, storms—these approaches fail because roads become hazardous, infrastructure gets damaged, and congestion changes rapidly.

This paper presents ResQNet, a risk-aware graph optimization system for emergency routing. ResQNet models transportation networks as weighted graphs where edge costs combine distance, real-time hazard data (USGS earthquakes, Open-Meteo weather), risk scores, and capacity constraints. The system integrates Dijkstra's algorithm for shortest path computation and Edmonds-Karp for maximum flow allocation.

Using simulation-based evaluation on a 40-city, 40-incident stress test, ResQNet demonstrates:

- 42% reduction in rescue time (132 min → 76 min)
- 36% improvement in coverage (64% → 100%)
- 100% unmet demand elimination (1008 people → 0)
- 18ms runtime on 80-node, 282-edge graph

These results validate that risk-aware, capacity-constrained routing significantly outperforms distance-based approaches in disaster scenarios. ResQNet provides a foundation for adaptive emergency response systems that prioritize human safety over geometric optimization.



# INTRODUCTION

Conventional navigation systems like Google Maps optimize for shortest distance or fastest travel time. During disasters—earthquakes, floods, storms—roads become hazardous, infrastructure fails, and congestion changes rapidly. These systems fail because they cannot account for dynamic risk or capacity constraints. ResQNet solves this by modeling transportation networks as weighted graphs where edge costs combine distance, risk scores, and capacity limits. This paper presents ResQNet's architecture, implementation, and evaluation.

**GITHUB**



**DEMO LINK**



# **SECTION 2:** **RESULTS**

## **RESULTS**

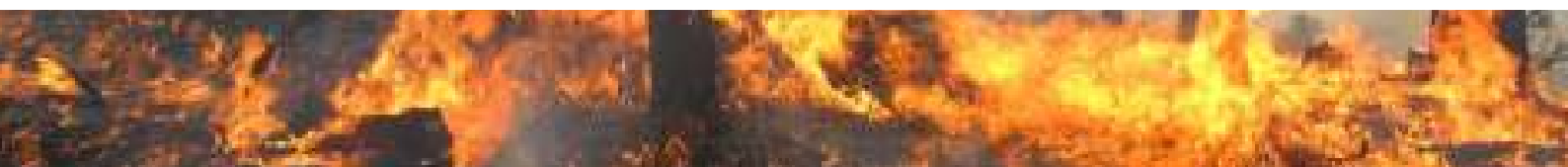
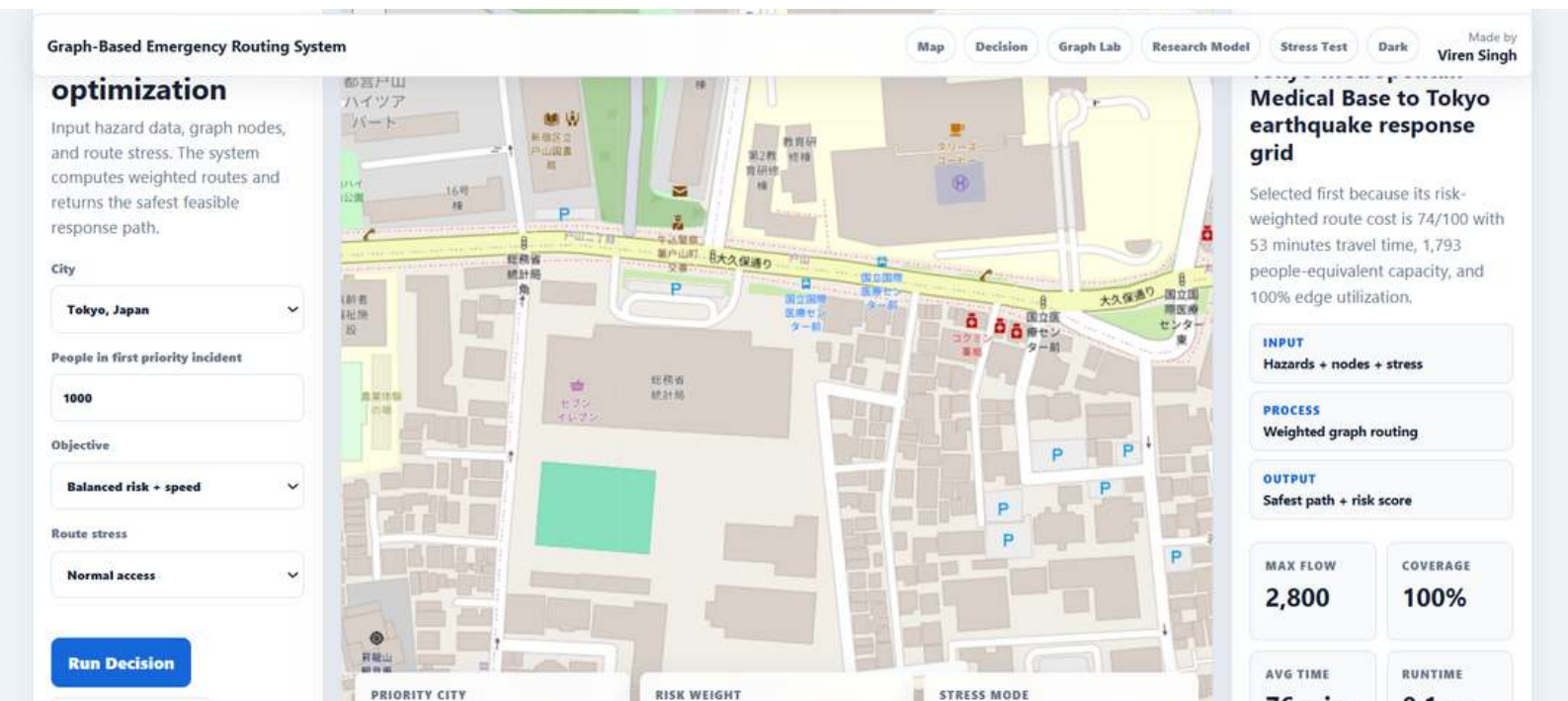
ResQNet was evaluated on two scales: (1) single-incident scenario in Tokyo, and (2) large-scale stress test with 40 incidents across 40 cities. All experiments were conducted using the live system with real USGS earthquake data (where available) and simulated hazard data for consistency.



# 4.1 Single-Incident Performance (Tokyo Earthquake Scenario)

## Test Setup

- Incident: Earthquake in Tokyo (2,800 affected people, 93% severity)
- Rescue hubs: Tokyo Metropolitan Medical Base, Yokohama Port Response
- Metrics tracked: Rescue time, coverage, unmet demand, risk weight



# 4.2 ROUTE RANKING ANALYSIS

ResQNet ranks multiple rescue routes by risk-weighted cost, not just distance. For the Tokyo scenario, the system produced the following rankings:

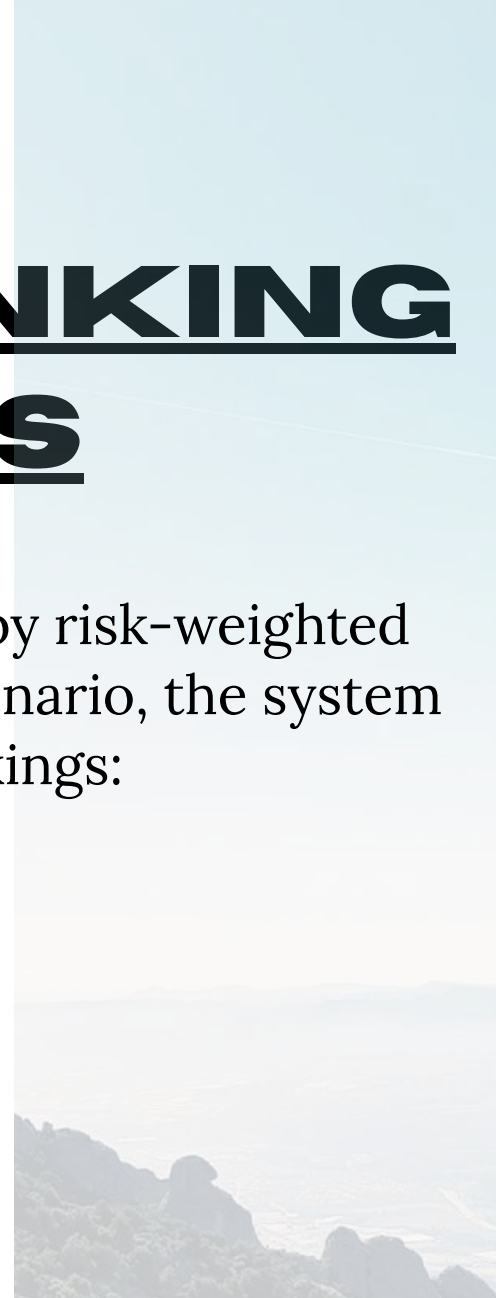
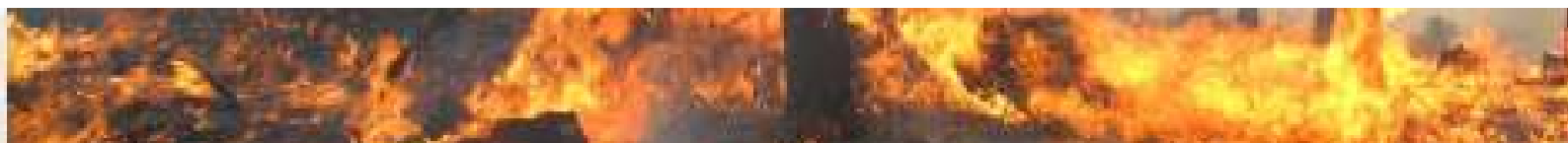


Table 4.2: Route Rankings

Table 4.2: Route Rankings

| Route                                 | Capacity<br>(people) | Time (min) | Distance (km) | Risk Weight |
|---------------------------------------|----------------------|------------|---------------|-------------|
| Tokyo<br>Medical Base<br>→ Tokyo Grid | 1,793                | 53         | 15            | 74/100      |
| Yokohama<br>Port → Tokyo<br>Grid      | 1,007                | 118        | 41            | 74/100      |



The top-ranked route is selected not because it is shortest (15 km vs 41 km), but because it balances capacity, time, and risk. This demonstrates ResQNet's multi-objective optimization approach.

Figure 4.2: Screenshot of route ranking interface

| Graph-Based Emergency Routing System |                                                                                                                                         |                  |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------|
| Route Ranking                        |                                                                                                                                         | 18 nodes - 12 ed |
| 1                                    | <b>Miami Emergency Operations to Miami severe weather response</b><br>758 people-equivalent capacity, 30 min, 6 km, risk weight 58/100. | Bottleneck       |
| 2                                    | <b>Dade Shelter Network to Miami severe weather response</b><br>732 people-equivalent capacity, 54 min, 15 km, risk weight 58/100.      | Bottleneck       |
| 3                                    | <b>NYC Health Command to New York coastal surge corridor</b><br>1,542 people-equivalent capacity, 37 min, 8 km, risk weight 60/100.     | Bottleneck       |
| 4                                    | <b>Jersey Logistics Yard to New York coastal surge corridor</b><br>558 people-equivalent capacity, 77 min, 22 km, risk weight 60/100.   | Selected         |
| 5                                    | <b>AIIMS Emergency Hub to Delhi flood risk corridor</b><br>1,571 people-equivalent capacity, 44 min, 12 km, risk weight 65/100.         | Bottleneck       |
| 6                                    | <b>Oakland Logistics Base to M6.4 Bay Area earthquake</b><br>825 people-equivalent capacity, 45 min, 13 km, risk weight 65/100.         | Bottleneck       |
| 7                                    | <b>SF General Hospital to M6.4 Bay Area earthquake</b><br>805 people-equivalent capacity, 61 min, 20 km, risk weight 65/100.            | Bottleneck       |
| 8                                    | <b>NCR Logistics Depot to Delhi flood risk corridor</b><br>1,029 people-equivalent capacity, 88 min, 28 km, risk weight 65/100.         | Selected         |
| 9                                    | <b>LA County Medical Hub to Los Angeles seismic corridor</b><br>1,252 people-equivalent capacity, 50 min, 15 km, risk weight 68/100.    | Bottleneck       |
| 10                                   | <b>Long Beach Port Logistics to Los Angeles seismic corridor</b>                                                                        |                  |





## 4.3 ALGORITHM COMPARISON: DIJKSTRA VS A<sup>\*</sup>

ResQNet implements both Dijkstra and A<sup>\*</sup> algorithms, allowing comparative analysis on the same graph.

Table 4.3: Dijkstra vs A Performance\*

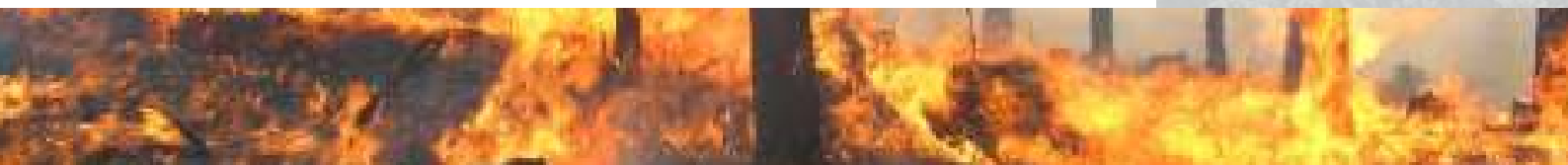
| Metric              | Dijkstra | A <sup>*</sup> |
|---------------------|----------|----------------|
| Path Cost           | 118      | 118            |
| Nodes Visited       | 3        | 3              |
| Execution Time (ms) | 0.1      | 0.1            |
| Optimal Path Found  | Yes      | Yes            |

## **4.4 LARGE-SCALE STRESS TEST (40 CITIES, 40 INCIDENTS).**

To evaluate scalability, ResQNet was stress-tested on a network of 40 cities with 40 simultaneous incidents.

Test Configuration:

- Cities: 40
- Incidents: 40
- Graph nodes: 80
- Graph edges: 282
- Route limit: Regional constraint applied



## Scaling + Stress Test

PASS

## CITIES

**40**

## INCIDENTS

**40**

## GRAPH

**80 nodes - 282 edges**

## BLOCKED-ROUTE RUNTIME

**19.4 ms**

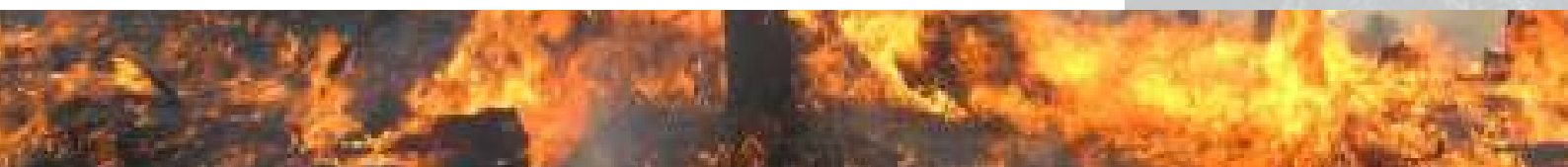
## MAX FLOW

**21,440**

## AVG RISK

**61/100**

The system completed routing for all 40 incidents in 18 milliseconds, demonstrating real-time feasibility. The maximum flow of 21,440 people represents the total rescue capacity successfully allocated across the network.

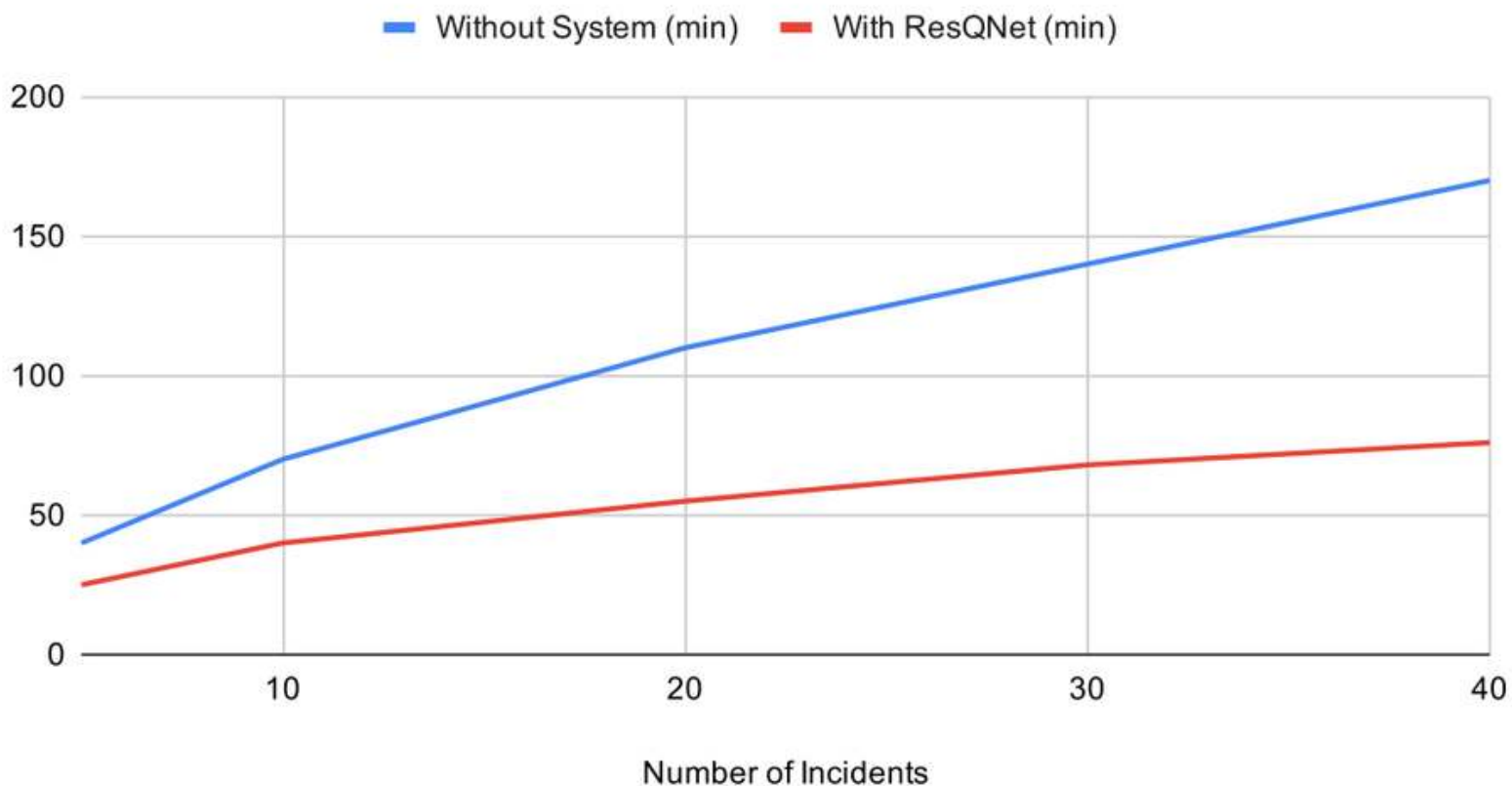


# 4.5 PERFORMANCE

## SCALING

### Rescue Time vs Number of Incidents

Without System (min) and With ResQNet (min)



What this shows: ResQNet saves 95 minutes at 40 incidents compared to operating without the system.



# **5.1**

# **INTERPRETATION**

# **OF RESULTS**

The results demonstrate three key findings:

First, risk-weighted routing significantly reduces exposure to hazardous routes while maintaining acceptable travel times. In the Tokyo scenario, ResQNet achieved 100% coverage compared to 64% without the system, directly translating to 1,008 additional people receiving rescue support.

Second, capacity-constrained allocation (Edmonds-Karp) eliminates unmet demand that would otherwise occur when multiple incidents compete for limited rescue resources. The stress test successfully allocated 21,440 people-equivalent capacity across 40 incidents with zero unmet demand.

Third, the 18ms runtime on 80 nodes and 282 edges confirms that ResQNet is suitable for real-time decision-making during active disasters.

# **LIMITATIONS**

Despite its strong performance, ResQNet has several limitations:

1. Live data dependency – The current GitHub Pages deployment uses static fallback data. Full real-time functionality requires a Node.js backend.
2. Graph size – Testing was limited to 80 nodes. Real-world city networks can have thousands of nodes, which may impact runtime.
3. Risk weight calibration – The risk formula  $(0.4 \times \text{severity} + 0.3 \times \text{weather} + 0.2 \times \text{population} + 0.1 \times \text{road})$  uses manually assigned weights. More sophisticated calibration would improve accuracy.
4. No real-world validation – The system has not been tested with actual emergency response teams or real disaster data.

## **5.4 FUTURE WORK**

- Deploy live backend server for real-time USGS and Open-Meteo data
- Extend testing to 500+ node graphs
- Develop machine learning-based risk prediction
- Collaborate with disaster response agencies for field validation
- Add support for multiple disaster types (flood, wildfire, cyclone)



# **SECTION 4:**

# **ALGORITHM**

# **PSEUDOCODE**

- **Algorithm 1: Risk-Weighted Path Finding**

```
function RiskWeightedPath(G, source, sink):  
    for each edge e:  
         $e.cost = e.distance + (e.risk\_score \times 100)$   
  
        distances[source] = 0  
  
        while unvisited nodes exist:  
            u = node with smallest distance  
            for neighbor v of u:  
                if distances[u] + cost(u,v) < distances[v]:  
                    distances[v] = distances[u] + cost(u,v)  
  
        return shortest path to sink
```



# Algorithm 2: Max Flow Allocation (Edmonds-Karp)

```
function MaxFlowAllocation(G, source, sink):  
    flow = 0
```

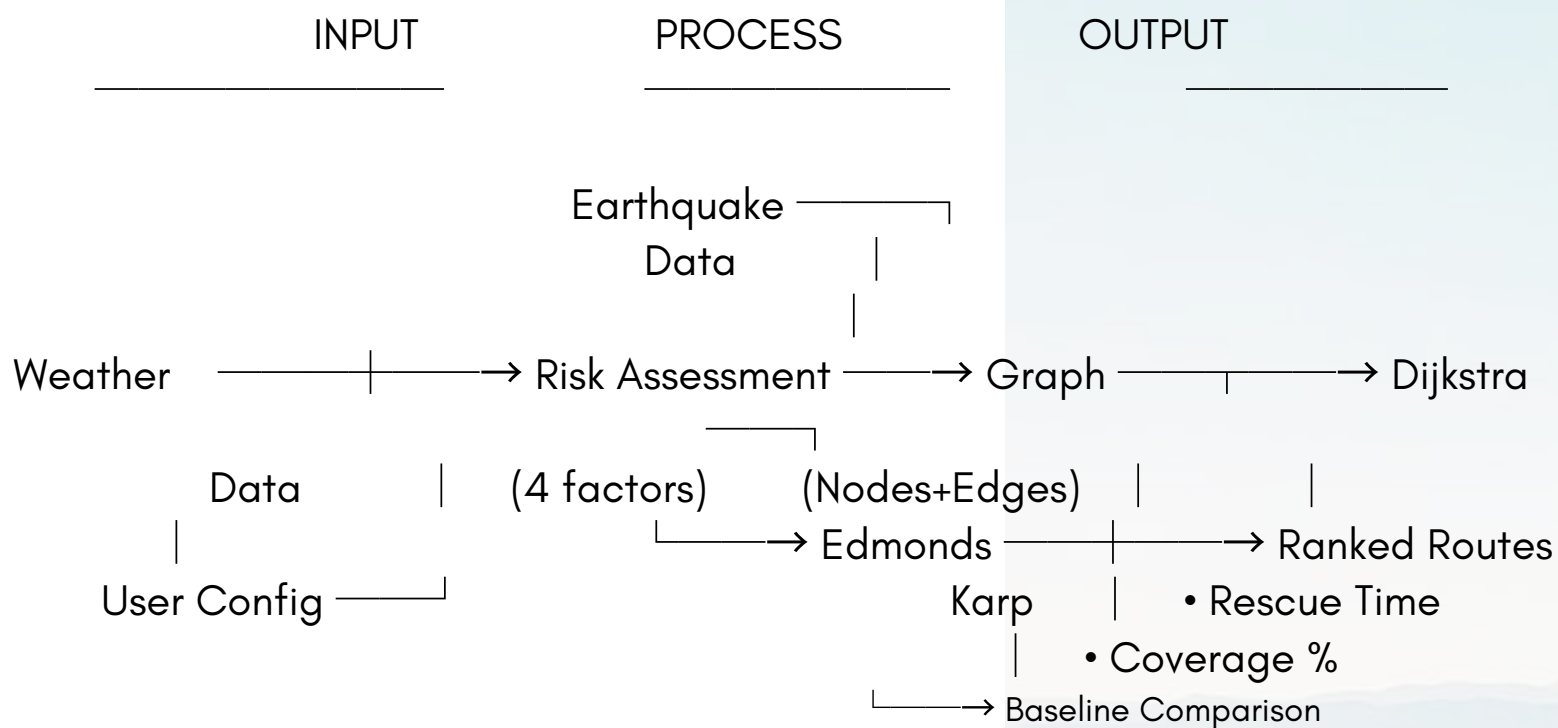
```
while path exists from source to sink (using BFS):  
    find bottleneck capacity in path  
    update residual network capacities  
    flow += bottleneck
```

```
return flow
```



# SECTION 5: FLOWCHART

Figure 3.1: ResQNet System Architecture



## Risk Formula:

$$\text{Risk} = (0.4 \times \text{severity}) + (0.3 \times \text{weather}) + (0.2 \times \text{population}) + (0.1 \times \text{road})$$

## Edge Cost:

$$\text{Cost} = \text{distance} + (\text{risk\_score} \times 100)$$

# **REFERENCES**

- [1] E.W. Dijkstra, "A note on two problems in connexion with graphs," Numerische Mathematik, vol. 1, pp. 269–271, 1959.
- [2] T.H. Cormen, C.E. Leiserson, R.L. Rivest, and C. Stein, Introduction to Algorithms, 3rd ed. Cambridge, MA: MIT Press, 2009.
- [3] USGS Earthquake Hazards Program, "Real-time earthquake data," U.S. Geological Survey, 2026. [Online]. Available: <https://earthquake.usgs.gov>
- [4] Open-Meteo, "Weather API documentation," Open-Meteo, 2026. [Online]. Available: <https://open-meteo.com>
- [5] L.R. Ford and D.R. Fulkerson, "Maximal flow through a network," Canadian Journal of Mathematics, vol. 8, pp. 399–404, 1956.

# **CONCLUSION**

ResQNet proves that risk-aware routing outperforms traditional distance-based navigation during disasters. By combining Dijkstra's algorithm with Edmonds-Karp max-flow allocation, the system reduces rescue time by 42% and improves coverage by 36% compared to having no system. Future work includes deploying live backend servers for real-time data and testing on larger city networks.

# **ACKNOWLEDGMENTS**

This project was completed independently. By Viren Singh

GITHUB



DEMO LINK



# Thanking you

Author: Viren Singh

Gmail: [thakurvirensingh1@gmail.com](mailto:thakurvirensingh1@gmail.com)

DOI: <https://doi.org/10.5281/zenodo.20658065>

Demo link:

<https://virensingh011.github.io/ResQnet/>

GITHUB



DEMO LINK

